

Data Structures and Algorithms

Data Structures and Algorithms (DSA)

Data Structures and Algorithms (DSA) are important concepts in computer programming. A data structure is a method of organizing and storing data so that a program can use it properly. An algorithm is a series of steps used to solve a problem or complete a specific task.

Data structures and algorithms are connected because an algorithm needs data to work with. The way data is organized can affect how quickly a program can find, change, or process information. Some common data structures include arrays, linked lists, stacks, queues, trees, and graphs (GeeksforGeeks, n.d.).

DSA is important because computer programs often work with large amounts of information. A good data structure can make information easier to manage, while an efficient algorithm can reduce the amount of time needed to complete a task.

For example, a school information system may contain student names, student numbers, grades, schedules, and other records. If these records are properly organized, the system can find and process information more efficiently. Learning DSA helps students understand how programs handle data and how different methods can affect program performance.

Linear Data Structure

A Linear Data Structure is a type of data structure where elements are arranged in a sequential order. Each element is connected to the next element in a logical sequence. Common examples include arrays, linked lists, stacks, and queues.

Linear data structures are useful when information needs to be processed in a specific order. Depending on the type of structure, elements can be added, removed, or accessed in different ways (OpenDSA, n.d.).

Examples of Linear Data Structures

Array

An array stores multiple elements in an ordered collection. Each element has a position called an index. In many programming languages, the first element has an index of 0.

Example:

[80, 85, 90, 75, 88]

The values can represent the grades of five students.

- Index 0 = 80
- Index 1 = 85
- Index 2 = 90
- Index 3 = 75
- Index 4 = 88

An array is useful when a program needs to store several related values and access them using their positions.

Linked List

A linked list is a collection of elements called nodes. Each node contains data and a reference or link to another node. Unlike an array, the elements of a linked list do not have to be stored next to each other in memory.

For example:

Student A → Student B → Student C → Student D

Each student record can point to the next record. This structure can be useful when elements need to be inserted or removed without requiring the entire collection to be rearranged.

Stack

A stack follows the **Last In, First Out (LIFO)** principle. This means the most recently added element is the first one removed.

A simple example is a stack of books. If you place Book A, then Book B, and finally Book C on top, Book C must be removed first.

Stacks are commonly used in programming for tasks such as keeping track of function calls and undo operations.

Queue

A queue follows the **First In, First Out (FIFO)** principle. This means the first element added to the queue is the first element removed.

A real-life example is a line of students waiting at a cashier. The student who arrives first is normally served first.

Queues can be used in computer systems to manage tasks, requests, or processes that need to be handled in order.

One-Dimensional (1D) Array

A One-Dimensional Array is a linear data structure that stores elements in a single sequence. Each element can be accessed using one index. It is useful when several values of the same general type need to be stored together (W3Schools, n.d.).

For example:

Product Prices = [120, 250, 175, 300, 450]

The array contains five product prices. The first price is located at index 0, while the last price is located at index 4.

A program can directly access a value if its index is known. For example:

Product Prices[2] = 175

This means that the value at index 2 is 175.

Advantages of a 1D Array

One advantage of a one-dimensional array is that it provides direct access to elements through their indexes. This makes it useful when a program frequently needs to retrieve values from known positions.

Arrays are also simple to understand and are commonly used in programming. They can be used to store information such as grades, prices, quantities, scores, or identification numbers.

Limitation of a 1D Array

One limitation of a traditional array is that its size may be fixed when it is created. If a program needs to store more elements than the available space, additional handling may be required.

Because of this, programmers should consider the amount of data that needs to be stored and whether the data will frequently change in size.

The Big O Notation

Big O Notation is used to describe how the performance of an algorithm changes as the amount of input data increases. It helps programmers understand how much time or computational resources an algorithm may require as the size of the problem grows (Cormen et al., 2022).

Big O does not focus on the exact number of seconds an algorithm takes on one computer. Instead, it focuses on how the amount of work grows when more data is added.

Common Big O Complexities

O(1) – Constant Time

O(1) means that an operation takes approximately the same amount of work regardless of the size of the input.

For example, accessing an array element using its index is generally O(1).

If a program wants to access:

Grades[4] can directly access the value at index 4 without checking the previous elements.

$O(\log n)$ – Logarithmic Time

$O(\log n)$ means that the amount of work grows slowly as the input size increases. An algorithm with this complexity usually reduces the amount of data it needs to process during each step.

Binary Search is a common example. Instead of checking every element, it repeatedly divides the search area into smaller parts.

$O(n)$ – Linear Time

$O(n)$ means that the amount of work increases directly with the number of elements.

For example, if a program checks every student record in a list to find a particular student, it may need to check each record. If the list becomes twice as large, the program may need approximately twice as many checks.

Linear Search has $O(n)$ worst-case time complexity.

$O(n^2)$ – Quadratic Time

$O(n^2)$ means that the amount of work can grow much faster as the input increases. It is commonly seen in algorithms that use nested loops.

For example, an algorithm may compare every item with many other items in the same collection. Basic Bubble Sort can have $O(n^2)$ worst-case time complexity.

Importance of Big O

Big O is useful because it helps programmers compare algorithms. Two algorithms may produce the same result but require different amounts of work.

For example, searching through a small list may not show a large difference between two methods. However, when the list contains thousands or millions of records, the difference in efficiency can become more important.

Understanding Big O can help programmers choose an appropriate algorithm when developing systems that handle large amounts of data.

Basic Searching Algorithms

Searching algorithms are used to locate a specific value or record inside a collection of data. Two common searching methods are Linear Search and Binary Search.

A. Linear Search

Linear Search is a simple searching algorithm that checks elements one at a time until the target value is found or the entire collection has been checked.

For example, consider this list:

[12, 25, 37, 48, 60]

If the program is searching for **48**, it checks:

12 → 25 → 37 → 48

Once 48 is found, the search can stop.

Advantages

Linear Search is easy to understand and does not require the data to be sorted. This makes it useful for small collections or situations where the data is not arranged in a particular order.

For example, if a small list of customer records is not sorted, Linear Search can still check the records one by one.

Disadvantages

The main disadvantage of Linear Search is that it can become slow when there are many elements. If the target value is at the end of the collection or does not exist, the algorithm may need to check every element.

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(n)$
- **Worst Case:** $O(n)$

The best case occurs when the target is the first element. The worst case occurs when the target is the last element or cannot be found.

B. Binary Search

Binary Search is an algorithm used to find a target value by repeatedly dividing a **sorted** collection into smaller sections.

For example, consider the sorted list:

[10, 20, 30, 40, 50, 60, 70]

Suppose the program is searching for **60**.

The algorithm first checks the middle value: **40**

Since 60 is greater than 40, the algorithm ignores the values smaller than 40.

It then searches:

[50, 60, 70]

The middle value is then checked again. The process continues until 60 is found.

Requirement

Binary Search requires the data to be sorted. If the data is not sorted, Binary Search cannot be directly applied correctly.

This requirement is important because the algorithm depends on the order of the elements to determine which half of the data can be ignored.

Advantages

Binary Search can be much faster than Linear Search when working with large, sorted collections. Each step removes approximately half of the remaining search area.

For this reason, Binary Search can be useful in systems that frequently search through large amounts of already sorted data.

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$

The logarithmic growth means that the number of steps increases slowly even when the size of the collection becomes much larger.

Linear Search vs. Binary Search

Feature	Linear Search	Binary Search
Method	Checks elements one at a time	Divides the search area into smaller sections
Sorted Data Required?	No	Yes
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Easy to Implement?	Yes	Yes, but requires additional conditions
Useful For	Small or unsorted data	Large and sorted data

Linear Search is useful when the data is small or not sorted. Binary Search can reduce the number of comparisons when the data is sorted.

The choice between the two depends on the situation. A programmer should consider the size of the data, whether the data is sorted, and how frequently the system needs to perform searches.

References

GeeksforGeeks. (n.d.). *Data structures*.

<https://www.geeksforgeeks.org/data-structures/>

OpenDSA. (n.d.). *Linear data structures*. Virginia Tech.

<https://opendsa-server.cs.vt.edu/>

W3Schools. (n.d.). *Data structures and algorithms*. <https://www.w3schools.com/dsa/>

W3Schools. (n.d.). *JavaScript arrays*. https://www.w3schools.com/js/js_arrays.asp